

PROJECT REPORT:

VEHICLE SERVICE COST CALCULATOR

Student Name : K S M KISHINDRA

Registration No : 26MEI10033

Repository URL : <https://github.com/kishindra26mei10033-boop/Vityarthi-Project>

1. Introduction:

The Vehicle Service Cost Calculator is a terminal-based command-line interface (CLI) application developed in Python. It is designed to automate and standardize the estimation and billing workflow for vehicle servicing centers. The system supports multiple vehicle tiers—specifically Premium Bikes, Normal Bikes, SUVs, and MUVs—and delivers transparent, itemized invoices incorporating statutory Goods and Services Tax (CGST and SGST).

2. Problem Statement:

In traditional multi-vehicle service setups, cost calculations are typically executed manually or through disparate notes, leading to calculation discrepancies, lack of price transparency, and omission of statutory tax breakdowns. Customers often encounter confusion regarding package pricing versus individual service charges. This project addresses these inefficiencies by delivering a deterministic, automated terminal application that enforces rate consistency and eliminates duplicate charges.

3. Scope of the Project:

In-Scope:

- Multi-category vehicle pricing (Premium Bike, Normal Bike, SUV, MUV).
- Service selection covering Brake Service, Engine Check, Oil Change, and Full Service.
- Automated duplicate entry prevention and full-service package overrides.
- Itemized invoicing with 18% GST (9% CGST + 9% SGST) breakdown.
- Pure terminal/CLI-based execution with invalid input handling.

Out-of-Scope:

- External SQL/NoSQL database persistence.
- Graphical User Interface (GUI) or web deployment.
- Live payment gateway processing.

4. Functional Requirements:

• Fn-1: Session Control:

Interactive main menu allowing users to either enter the calculator program or cleanly exit.

• Fn-2: Rate Table Display:

Presents a rate comparison grid via tabulate upon initialization.

• Fn-3: Category & Service Selection:

Enables sequential selection of vehicle category and service requirements through numeric prompts.

- **Fn-4: Duplicate Detection:**

Identifies whether an individual service has already been added in the active session and prevents re-addition.

- **Fn-5: Override Logic:**

When 'Full Service' is selected, previous individual services are cleared, bundle pricing is applied, and invoice is finalized.

- **Fn-6: Invoicing & Tax Computation:**

Calculates subtotal, 9% CGST, 9% SGST, and total payable charges.

5. Non-Functional Requirements:

- **NFn-1 (Performance):** Zero computational lag; menu transitions and bill calculations execute in <100ms.

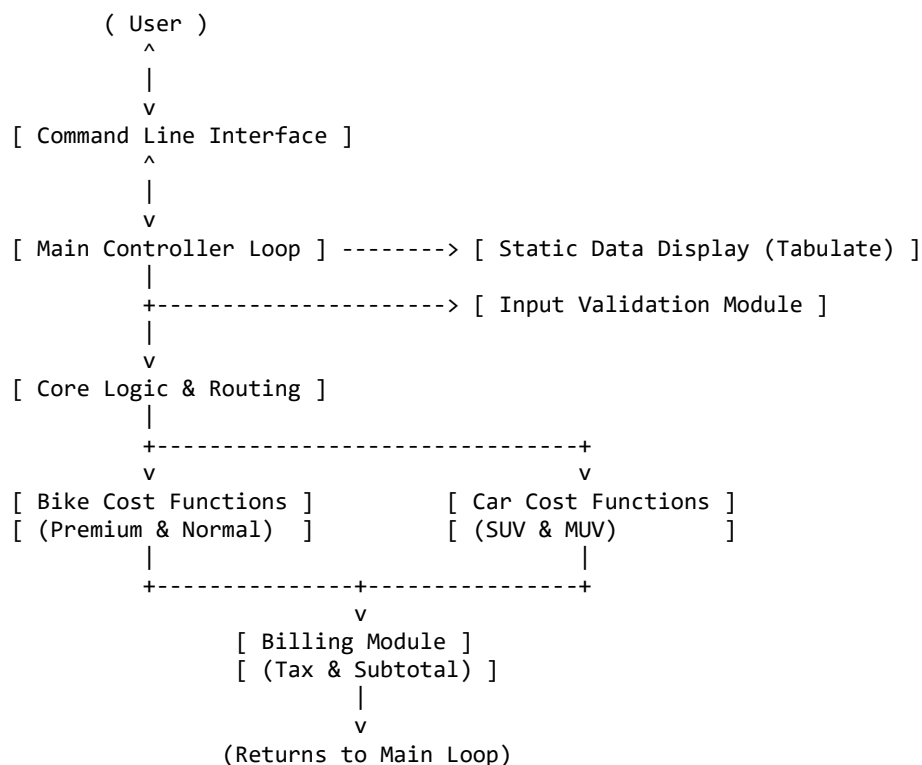
- **NFn-2 (Usability):** Numbered menu navigation minimizes typographical input errors.

- **NFn-3 (Reliability & Robustness):** Input range checks prevent out-of-bounds selections and script crashes.

- **NFn-4 (Maintainability):** Modular functional decomposition cleanly decouples pricing tables, selection logic, and bill printing.

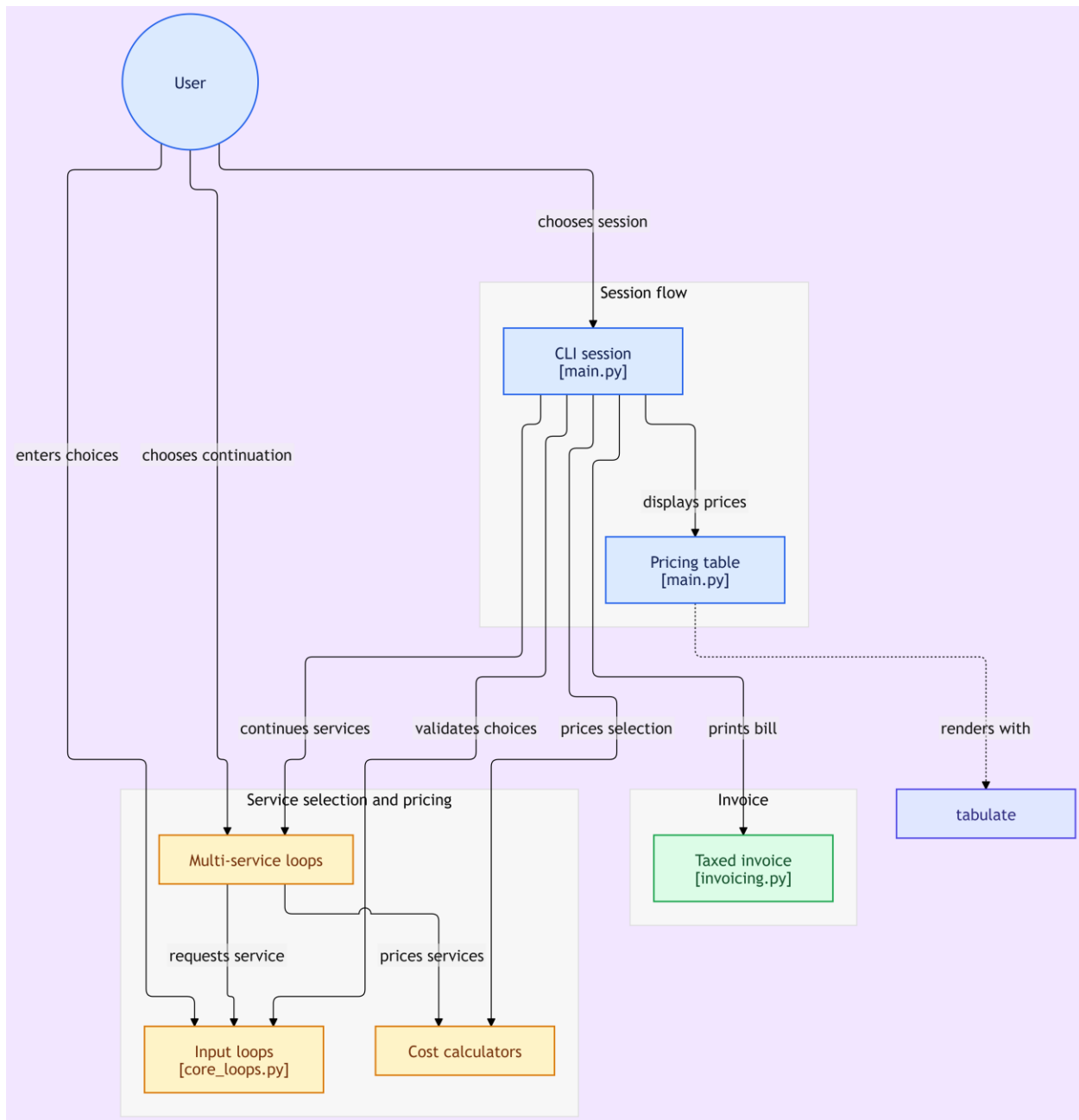
6. System Architecture:

The application operates as a 4-tier modular pipeline within the terminal runtime environment:

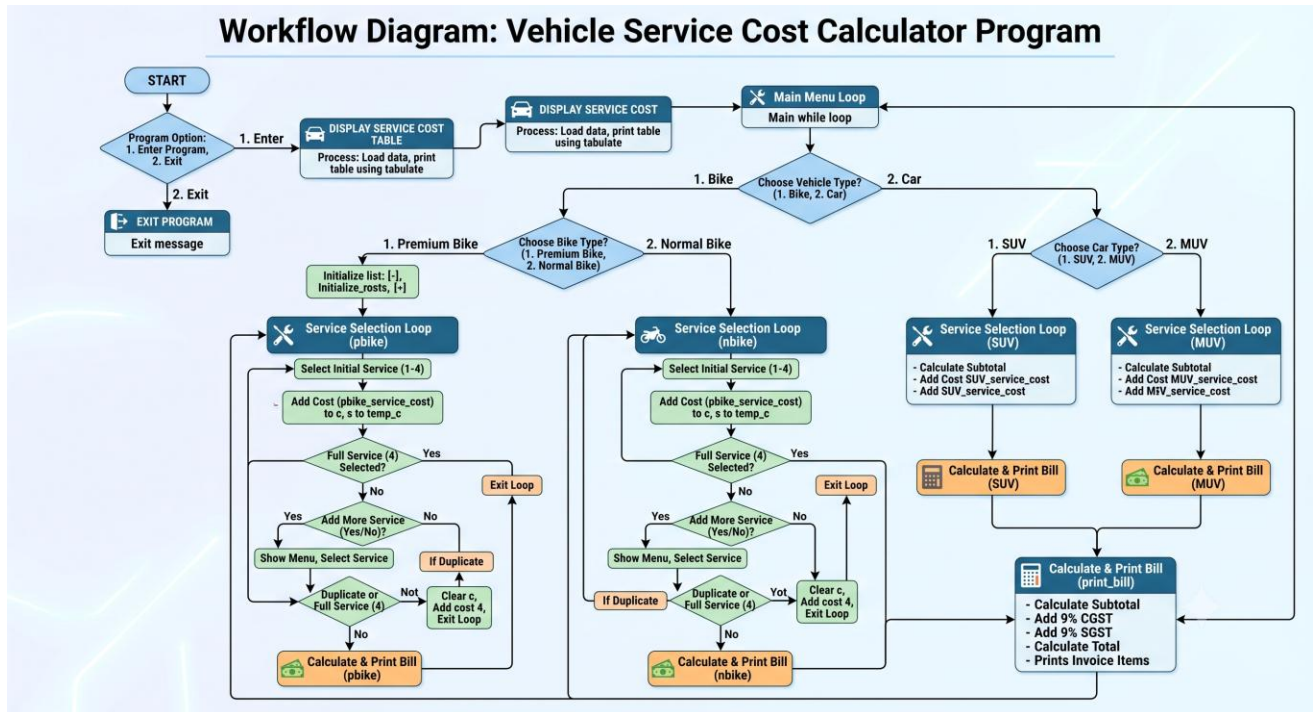


7. Design Diagrams:

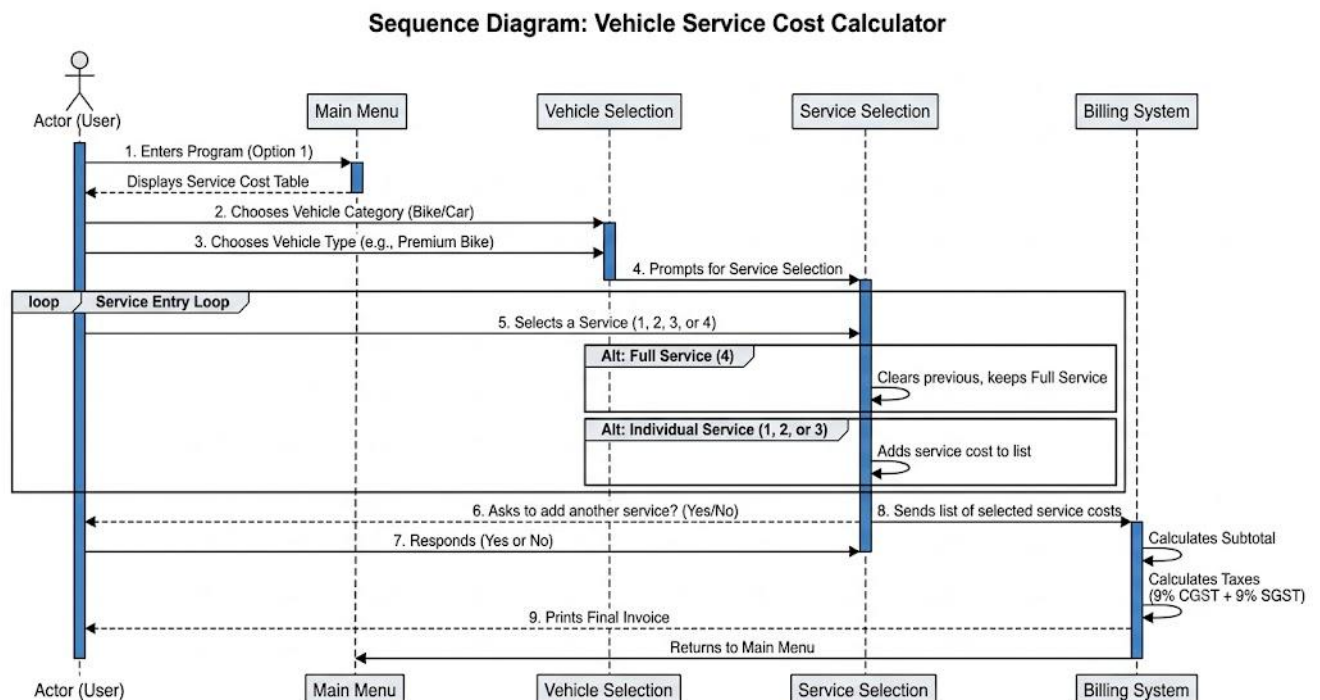
7.1 Use Case Diagram



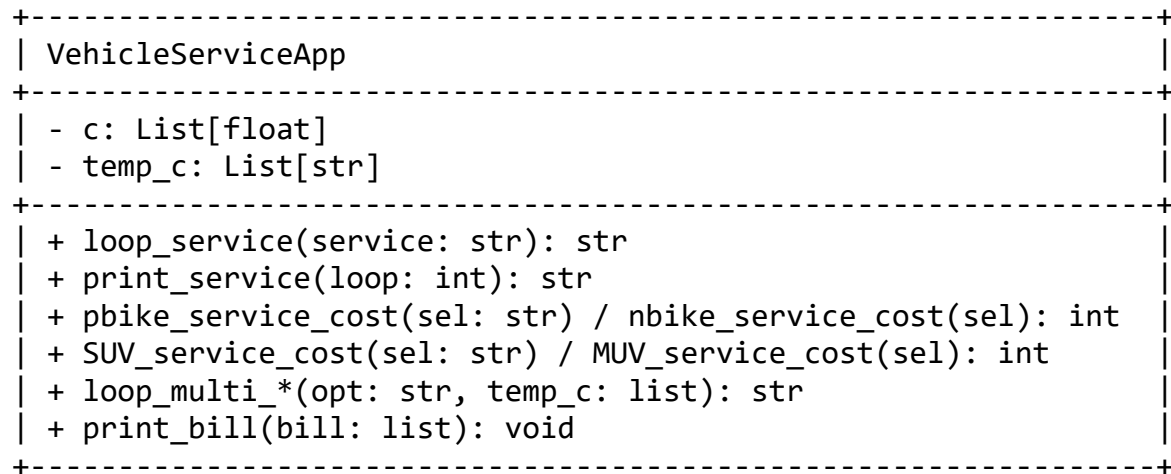
7.2 Workflow Diagram



7.3 Sequence Diagram

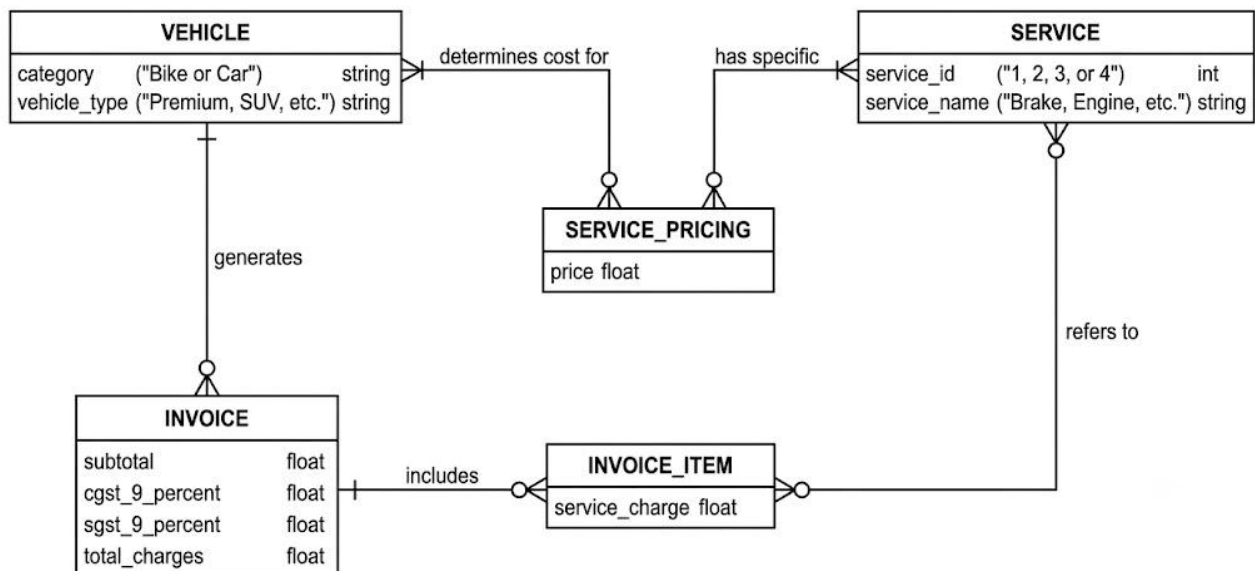


7.4 Class / Component Diagram



7.5 Entity Relationship (Data Model)

ENTITY RELATIONSHIP DIAGRAM: VEHICLE SERVICE COST CALCULATOR (Data Model)



8. Pricing Table:

PRICE	Premium bike	Normal bike	SUV	MUV
brake Service	700	300	9000	5000
engine check	500	200	3000	1200
oil change	1000	400	12000	8000
full service	2500	1000	25000	15000

9. Design Decisions & Implementation Details:

- **Procedural Modularization:** Vehicle-specific cost retrieval functions isolate pricing rules, enabling targeted updates without affecting other classes.
- **Duplicate Suppression:** Maintains an active tracking array (temp_c) verifying membership before appending fees to prevent duplicate entries.
- **Package Override:** When Option 4 (Full Service) is invoked, prior individual service charges are cleared to prevent redundant billing.
- **Statutory Dual-Tax Compliance:** Applies 9% CGST and 9% SGST to meet Indian commercial tax compliance standards.

10. Results & Invoicing Output:

Sample terminal invoice output generated for Premium Bike (Brake Service + Oil Change):

```
=====
                INVOICE ITEMS
-----
service charge:  700.00
service charge: 1000.00
=====
Subtotal:       1700.00
      + CGST: 153.00
      + SGST: 153.00
-----
TOTAL CHARGES: 2006.00
=====
```

11. Test:

Test1: Menu Start

Input: 1

Expected Output: Displays pricing grid and vehicle prompt

```
=====
# Vehicle service cost calculator #
=====

-----
1. Enter the program
2. Exit
-----

Select your option (1 or 2): 1

+-----+-----+-----+-----+
| PRICE          | Premium bike | Normal bike | SUV | MUV |
+=====+=====+=====+=====+
| brake Service  | 700          | 300         | 9000 | 5000 |
+-----+-----+-----+-----+
| engine check   | 500          | 200         | 3000 | 1200 |
+-----+-----+-----+-----+
| oil change     | 1000         | 400         | 12000 | 8000 |
+-----+-----+-----+-----+
| full service   | 2500         | 1000        | 25000 | 15000 |
+-----+-----+-----+-----+

# Vehicle choice
1. Bike
2. Car
```

Status: Passed

Test2: Menu Exit

Input: 2

Expected Output: termination & greeting

```
-----
1. Enter the program
2. Exit
-----

Select your option (1 or 2): 2

Exiting the Vehicle Service Calculator. Thank you for using the program, Have a great day!
```

Status: Passed

Test3: Validation**Input:** 99**Expected Output:** Prints 'invalid input' and prompts again

```
# Vehicle choice
1. Bike
2. Car

Select your Option in numbrs:99
invalid input

Select your Option in numbrs:|
```

Status: Passed**Test4:** Validation**Input:** 1, then 1**Expected Output:** Warns duplicate service added

```
# Service required
1. brake Service
2. engine check
3. oil change
4. full service

Select your Option in numbrs:1

do you want to add more service (yes or no):yes

# Service required
1. brake Service
2. engine check
3. oil change
4. full service

Select your Option in numbrs:1
You have already added this service!

do you want to add more service (yes or no):|
```

Status: Passed

Test5: Package Override

Input: 1, then 4

Expected Output: Clears list, sets 2500, proceeds to bill

```
# Service required
1. brake Service
2. engine check
3. oil change
4. full service

Select your Option in numbrs:1

do you want to add more service (yes or no):yes

# Service required
1. brake Service
2. engine check
3. oil change
4. full service

Select your Option in numbrs:4

Full service selected. Replacing previous services and proceeding to bill.

=====
                INVOICE ITEMS
-----
service charge:  2500.00
=====
Subtotal:        2500.00
                + CGST: 225.00
                + SGST: 225.00
-----
TOTAL CHARGES: 2950.00
=====
-----
1. Enter the program
2. Exit
-----
Select your option (1 or 2): |
```

Status: Passed

12. Challenges Faced:

1. Handling non-integer and out-of-range user inputs gracefully without interrupting terminal execution.
2. Managing state & Data transitions in nested loops when users selects multiple services.
3. Designing an override mechanism that properly resets itemized costs when switching to full servicing.

13. Learnings & Key Takeaways:

- Applied practical control structures, functions, and loop sanitization in Python.
- Designed structured test cases and validated terminal workflows against academic rubrics.
- Gained experience in transparent software financial modeling and dual-tax computation.

14. Future Enhancements:

- File Persistence: Exporting generated bills automatically to local text or CSV files.
- Dynamic Configuration: Loading vehicle rate cards from external JSON or SQLite tables.
- Parts Catalog: Integrating itemized spare parts costs alongside labor service charges.

15. References:

1. Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
2. Astanin, S. Tabulate: Pretty-print tabular data in Python. <https://pypi.org/project/tabulate/>
3. Program Download links:
Python: <https://www.python.org/downloads/>
Git: <https://git-scm.com/install/windows>